Add Binary

```c
#include <string.h>
#include <stdlib.h>

char* addBinary(char* a, char* b) {
    int lenA = strlen(a);
    int lenB = strlen(b);
    int maxLen = lenA > lenB ? lenA : lenB;

    char* result = (char*)malloc((maxLen + 2) * sizeof(char));
    result[maxLen + 1] = '\0';

    int i = lenA - 1;
    int j = lenB - 1;
    int k = maxLen;
    int carry = 0;

    while (i >= 0 || j >= 0 || carry) {
        int sum = carry;
        if (i >= 0) sum += a[i--] - '0';
        if (j >= 0) sum += b[j--] - '0';

        result[k--] = (sum % 2) + '0';
        carry = sum / 2;
    }

    if (k == 0) {
        memmove(result, result + 1, maxLen + 1);
    }

    return result;
}
```

693. Binary Number with Alternating Bits

```c
#include <stdbool.h>

bool hasAlternatingBits(int num) {
    long n = num ^ (num >> 1);
    return (n & (n + 1)) == 0;
}
```

898. Bitwise ORs of Subarrays

```c
#include <stdlib.h>

int compare(const void* a, const void* b) {
    return (*(int*)a - *(int*)b);
}
```

```c
int subarrayBitwiseORs(int* arr, int arrSize) {
    int* vals = (int*)malloc(arrSize * 33 * sizeof(int));
    int totalSize = 0;
    int prevStart = 0;

    for (int i = 0; i < arrSize; i++) {
        int a = arr[i];
        int prevEnd = totalSize;

        vals[totalSize++] = a;
```

```
        for (int j = prevStart; j < prevEnd; j++) {
            int newVal = vals[j] | a;
            if (vals[totalSize - 1] != newVal) {
                vals[totalSize++] = newVal;
            }
        }

        prevStart = prevEnd;
    }

    qsort(vals, totalSize, sizeof(int), compare);

    int uniqueCount = (totalSize > 0) ? 1 : 0;
    for (int i = 1; i < totalSize; i++) {
        if (vals[i] != vals[i - 1]) {
            uniqueCount++;
        }
    }

    free(vals);
    return uniqueCount;
}
```